__/__/____

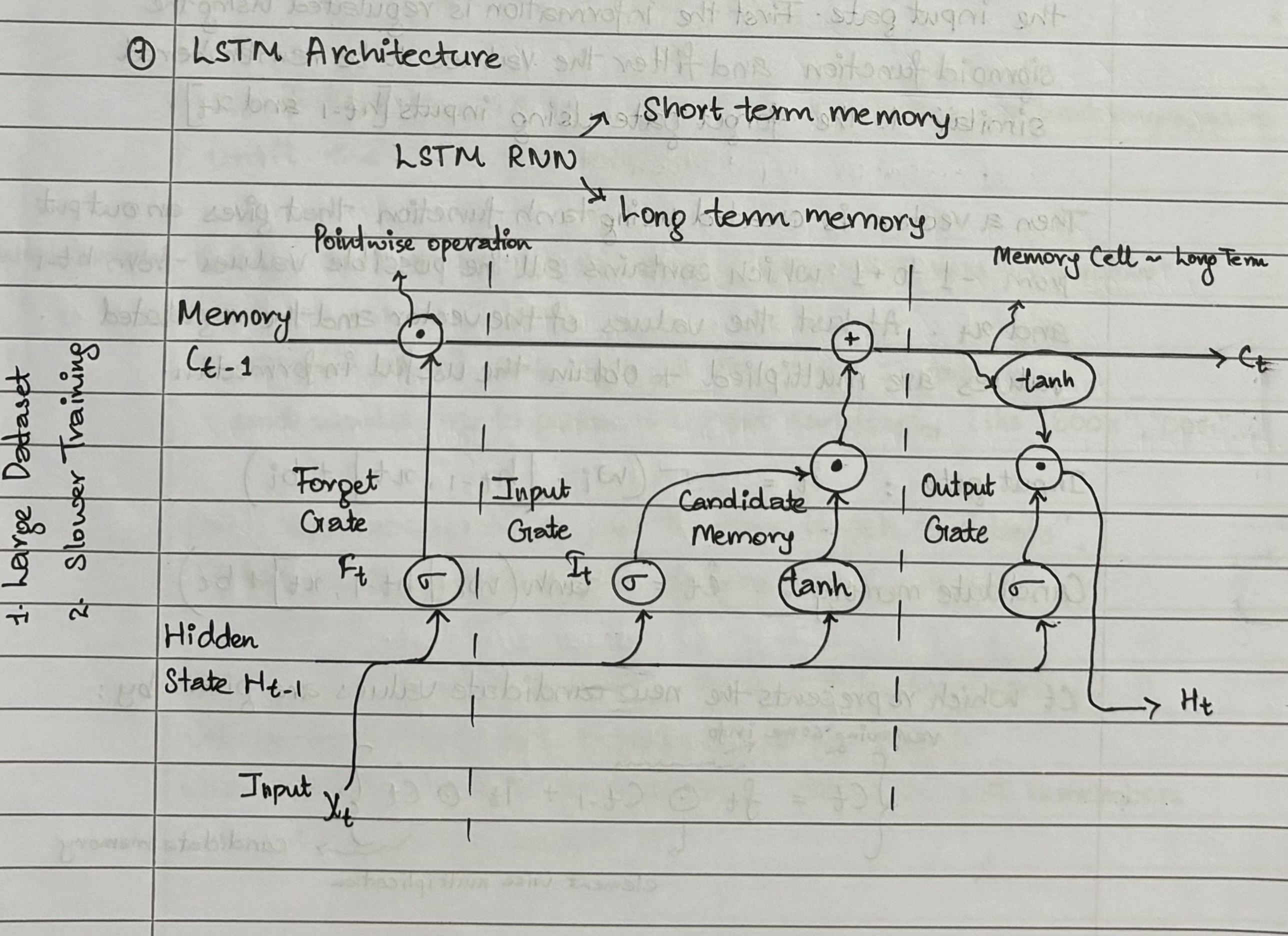
° LONG SHORT TERM MEMORY (LSTM-RNN)

LSTMs can capture long-term dependencies in sequential data making them ideal for tasks like language translation, speech recognition and time series forecasting.
LSTMs introduce a memory cell that holds information over extended periods addressing the challenge of learning long-term dependencies.

→ Problem with Long-Term Dependencies in RNN

Learning long term dependencies where information from distant time steps becomes crucial for making accurate predictions for current state. This is vanishing gradient or exploding gradient.

① LSTM Architecture



__/__/____

what % of long-term memory is remembered

1. Forget Gate :: The information that is no longer useful in the cell state is removed with the forget gate.
Two inputs x_t and h_{t-1} are fed to the gate and multiplied with weights and bias addition. The resultant is passed through sigmoid activation function which gives output in rage of [0, 1].

If output is 0, or near to 0 → information is forgotten.
If output is 1 or near to 1 → information is retained.

$$\left\{ f_t = \sigma\left(w_f \cdot \underbrace{\left[h_{t-1}, x_t \right]}_{\text{concatenation}} + b_f \right) \right\}$$

how we should update the long term memory

2. Input Gate and Candidate Memory:
The addition of useful information to the cell state is done by the input gate. First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs $[h_{t-1}$ and $x_t]$.

Then a vector is created using tanh function that gives an output from -1 to +1 which contains all the possible values from h_{t-1} and x_t. At last the values of the vector and the regulated values are multiplied to obtain the useful information.

Input gate : $i_t = \sigma\left(w_i \cdot \left[h_{t-1}, x_t \right] + b_i \right)$

Candidate memory : $\hat{C}_t = \tanh\left(w_c \left[h_{t-1}, x_t \right] + b_c \right)$

C_t which represents the new candidate values are given by:

removing some info

$$\left\{ C_t = \underbrace{f_t \odot C_{t-1}}_{\text{element wise multiplication}} + \underbrace{i_t \odot \hat{C}_t}_{\text{candidate memory}} \right\}$$

__/__/____

What % of short term memory to pass

3. Output Gate : It is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this timestep. First the gate uses a sigmoid function to determine which information from the current cell state will be the output. This is done using the previous hidden state h_{t-1} and the current input x_t:

$$\{o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)\}$$

Next the current cell state C_t is passed through a tanh activation function to scale its values between (-1 and +1)

Finally this transformed cell state is multiplied element-wise with o_t to produce the hidden state h_t :

$$\{h_t = o_t \odot \tanh(C_t)\}$$

we will keep on updating $[W_i, W_c, W_o]$ through backpropagation until the loss gets reduced.

WALKTHROUGH "SHE BOUGHT TICKETS BECAUSE THE CONCERT WAS TOMORROW"

n: gram (5: gram) only looks at the first 5 tokens and would try to guess the next randomly like "BOOK","DOG"...

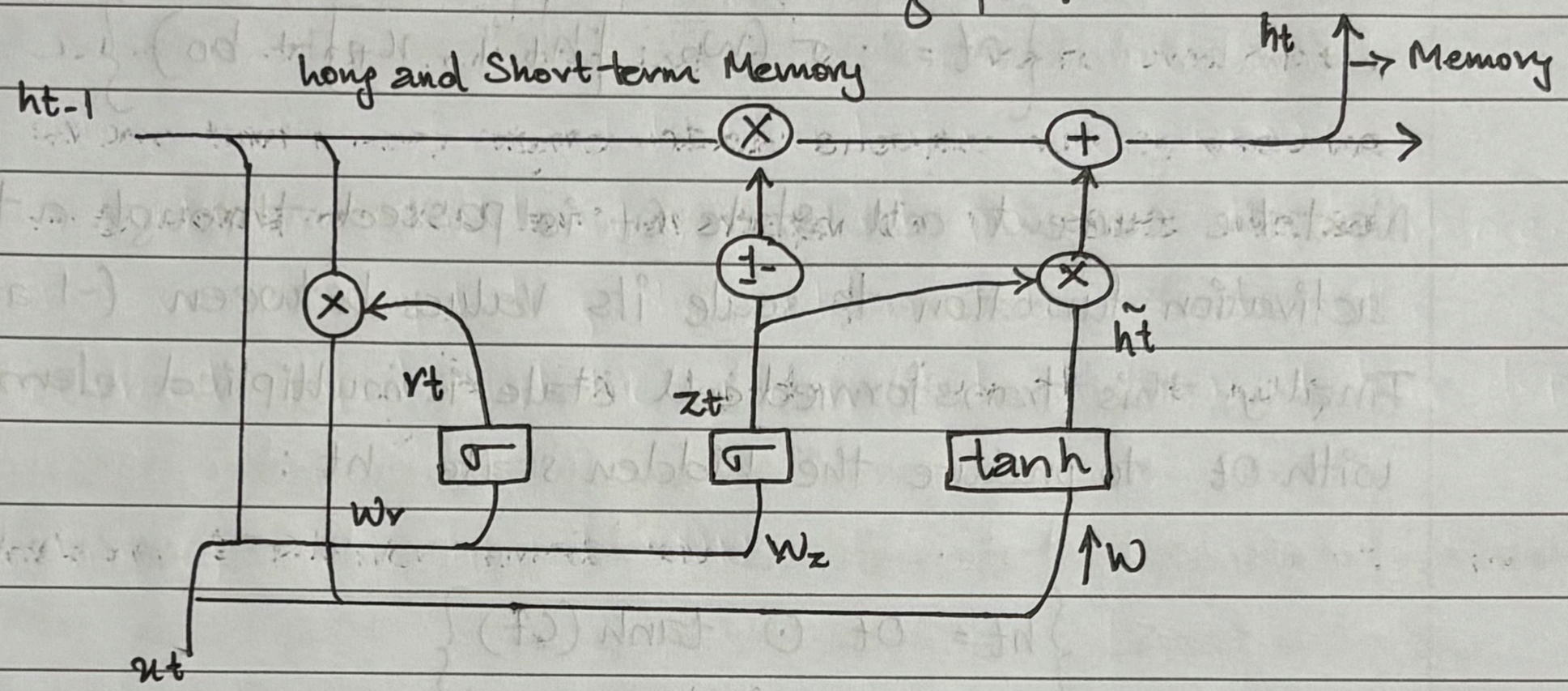
RNN read sequentially but the time it gets to "tickets", it has mostly forgotten other tokens like "concert".

LSTM, when it sees "CONCERT", it store it in the memory. Unimportant words are forgotten ("WAS","THE"). When predicting after "BECAUSE THE", the LSTM still remembers "concert" correctly predicts it.

__/__/____

6. GATED RECURRENT UNIT (GRU-RNN)

LSTMs are very complex structure with higher comptational cost. To overcome this Gated Recurrent Unit where introduced which uses LSTM architecture by merging its gating mechanisms offerering a more efficient solution for many sequential tasks without sacrificing performance.



1. Smaller Dataset
2. Faster Training

zt = Update Gate | Rt = Reset Gate

h̃t = Temporary Hidden State

1. Update Gate: This gate decides how much information (xt) from previous hidden state should be retained for the next time step.

$$\{ z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right) \}$$

2. Reset Gate: This gate determines how much of the past hidden state should be forgotten.

$$\{ r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right) \}$$

__/__/____

3. Candidate hidden state

$$\{\tilde{h}t = \tanh(Wh \cdot [rt \cdot ht-1, xt])\}$$

This is the potential new hidden state calculated based on the current input and previous hidden state.

4. Hidden State : The final hidden state is a weighted average of the previous hidden state ($ht-1$) and the candidate hidden state ($\tilde{h}t$) based on the update gate (zt)

$$\{ht = (1 - zt) \cdot ht-1 + zt \cdot \tilde{h}t\}$$

→ How GRUs solve the Vanishing Gradient Problem?

GRUs help mitigate this issue by using gates that regulate the flow of gradients during training ensuring that important information is preserved and that gradients do not shrink excessively over time.

__/__/____

• BIDIRECTIONAL RNN (BRNN)

It is an extension of the traditional RNN that processes sequential data in both forward and backward directions. This allows the network to utilize both past and future context when making predictions providing a more comprehensive understanding of the sequence.

→ Along with a forward hidden layer, the BRNN has a backward hidden layer which processes the sequence in reverse, updating the hidden state based on the current input and the hidden state of the next time step.

→ Working of Bidirectional RNN

1. Inputting a sequence.

2. Dual Processing
   - ↳ Forward direction: Current input + previous hidden state
   - ↳ Backward direction: Current input + next hidden state

3. Computing the hidden state :- A non linear activation function is applied to the weighted sum of the input and the previous hidden state creating a memory mechanism that allows the network to retain information from previous steps.

4. Determining the output.